

Gestion avancée des flux Node.js

Back-pressure, messaging et communication inter-process

Module 5 / 7

Objectifs du module

Durée : 3h30

- Gérer la back-pressure dans les flux Node.js et comprendre la régulation automatique
- Intégrer AMQP dans Node.js avec `amqp-lib` (RabbitMQ) et le Pub/Sub avec Redis
- Mettre en place la communication inter-process en temps réel entre instances

À l'issue de ce module, vous saurez découpler une API d'un worker via une file de messages et diffuser des événements à plusieurs instances Express.

Plan du module

1. **Back-pressure** : rappel des streams, `write()` qui retourne `false`, l'événement `drain`, `pipeline` qui régule automatiquement
2. **AMQP dans Node.js** : `amqplib`, exchanges, queues, routing keys, ack/nack, prefetch, dead-letter, RabbitMQ
3. **Pub/Sub avec Redis** et communication inter-process en temps réel : `ioredis`, SUBSCRIBE/PUBLISH, multi-instances, IPC, WebSockets, comparaison Redis/AMQP

1. Back-pressure

Le problème : producteur plus rapide que consommateur

La back-pressure est le mécanisme qui empêche un producteur rapide de saturer la mémoire d'un consommateur lent.

Sans régulation

- Le producteur écrit sans attendre
- Le buffer interne grossit indéfiniment
- La RAM explose, le process meurt

Avec back-pressure

- Le consommateur signale qu'il est plein
- Le producteur fait une pause
- La mémoire reste bornée

Chaque stream `Writable` possède un `highWaterMark` (16 Ko par défaut pour les buffers) : le seuil au-delà duquel il demande de ralentir.

`write()` qui retourne `false` et l'événement `drain`

```
function ecrireDonnees(writable, source) {
  let i = 0;
  function ecrire() {
    let ok = true;
    while (i < source.length && ok) {
      const dernier = i === source.length - 1;
      // write() retourne false quand le buffer interne dépasse highWaterMark
      ok = writable.write(source[i], dernier ? finir : undefined);
      i++;
    }
    if (i < source.length && !ok) {
      // On attend l'événement 'drain' avant de reprendre
      writable.once("drain", ecrire);
    }
  }
  function finir() { console.log("Tout est écrit"); }
  ecrire();
}
```

Ignorer le `false` de `write()` est la cause n°1 des fuites mémoire sous charge.

pipeline règle automatiquement

```
const { pipeline } = require("node:stream/promises");
const fs = require("node:fs");
const zlib = require("node:zlib");

async function compresser(entree, sortie) {
  await pipeline(
    fs.createReadStream(entree),
    zlib.createGzip(),
    fs.createWriteStream(sortie)
  );
  console.log("Pipeline terminé sans saturation mémoire");
}
```

`pipeline` gère pour vous :

- La back-pressure entre chaque étage
- La propagation des erreurs

- La fermeture des streams
- Le nettoyage en cas d'échec

Préférez-le toujours à `.pipe()` enchaîné.

2. AMQP dans Node.js

Le modèle AMQP : exchanges, queues, routing keys

Un producteur ne connaît jamais les queues : il publie sur un **exchange** qui route vers les queues selon une **routing key**.

Types d'exchange

- `direct` : routing key exacte
- `topic` : motifs (`commande.*`)
- `fanout` : diffusion à toutes les queues
- `headers` : selon les en-têtes

Flux

1. Producteur → exchange
2. Binding (routing key) → queue
3. Queue → consommateur

Le découplage est total : ni le producteur ni le consommateur ne se connaissent.

Publier avec amqplib

```
const amqp = require("amqplib");

async function publier() {
  const conn = await amqp.connect("amqp://localhost:5672");
  const ch = await conn.createChannel();
  await ch.assertExchange("commandes", "direct", { durable: true });

  const message = Buffer.from(JSON.stringify({ id: 42, total: 99.9 }));
  ch.publish("commandes", "commande.creee", message, {
    persistent: true, // message survit au redémarrage du broker
    contentType: "application/json",
  });
  console.log("Message publié");
  await ch.close();
  await conn.close();
}
```

Consommer : ack, nack et prefetch

```
async function consommer() {
  const conn = await amqp.connect("amqp://localhost:5672");
  const ch = await conn.createChannel();
  await ch.assertQueue("traitement", { durable: true });
  await ch.bindQueue("traitement", "commandes", "commande.creee");

  // prefetch = back-pressure AMQP : 1 seul message non acquitté à la fois
  await ch.prefetch(1);

  ch.consume("traitement", async (msg) => {
    try {
      const data = JSON.parse(msg.content.toString());
      await traiter(data);
      ch.ack(msg); // acquittement : message retiré de la queue
    } catch (err) {
      ch.nack(msg, false, false); // rejet sans requeue -> dead-letter
    }
  });
}
```

Dead-letter et durabilité

Dead-letter queue (DLQ)

- Reçoit les messages rejetés (`nack` sans requeue) ou expirés
- Configurée via `x-dead-letter-exchange`
- Permet l'analyse et le rejeu des échecs

Durabilité

- `durable: true` sur exchange et queue
- `persistent: true` sur le message
- Les trois sont nécessaires pour survivre à un crash du broker

```
await ch.assertQueue("traitement", {
  durable: true,
  arguments: {
    "x-dead-letter-exchange": "dlx",
    "x-dead-letter-routing-key": "commande.echec",
  },
});
```

Démo : Redis Pub/Sub multi-instances

Voir `demos/demo-05.md`

Deux instances Express partagent des notifications temps réel via Redis Pub/Sub.
Docker requis pour Redis.

3. Pub/Sub Redis et inter-process

SUBSCRIBE / PUBLISH avec ioredis

```
const Redis = require("ioredis");

// Une connexion en mode subscribe ne peut plus faire d'autres commandes
const abonne = new Redis();
const editeur = new Redis();

abonne.subscribe("notifications", (err, count) => {
  if (err) return console.error(err);
  console.log(`Abonné à ${count} canal/canaux`);
});

abonne.on("message", (canal, message) => {
  console.log(`[${canal}] ${message}`);
});

// Depuis n'importe quelle autre instance :
editeur.publish("notifications", JSON.stringify({ user: 7, texte: "Bonjour" }));
```

Multi-instances : diffuser à tout le cluster

Avec plusieurs instances Express derrière un load-balancer, un événement reçu par l'instance A doit atteindre les clients connectés à l'instance B.

Sans Redis

- Chaque instance est isolée
- Un WebSocket sur l'instance A ne voit pas un événement de l'instance B

Avec Redis Pub/Sub

- Chaque instance s'abonne au même canal
- Une instance publie, toutes reçoivent
- Chaque instance relaie à ses propres clients

C'est le principe derrière l'adaptateur Redis de Socket.IO.

IPC entre process Node et WebSockets

IPC avec child_process / cluster

```
const cp = require("node:child_process");
const enfant = cp.fork("./worker.js");
enfant.on("message", (m) =>
  console.log("Reçu", m)
);
enfant.send({ tache: "calcul" });
```

Communication directe entre process d'une même machine.

WebSockets

- Canal bidirectionnel client ↔ serveur
- Temps réel vers le navigateur
- Combiné à Redis pour le multi-instances

Comparaison Redis Pub/Sub vs AMQP

Critère	Redis Pub/Sub	AMQP (RabbitMQ)
Persistance des messages	Non (fire-and-forget)	Oui (durable + persistent)
Acquittement	Non	Oui (ack/nack)
Garantie de livraison	Au plus une fois	Au moins une fois
Routing	Canaux / patterns	Exchanges + routing keys
Cas d'usage	Notifications temps réel	Traitement de tâches fiable

Redis pour la diffusion volatile rapide, AMQP pour le découplage fiable de traitements.

Travaux pratiques

Fichier : `tp/tp-05.md`

Découpler une API Express d'un worker de traitement via une file AMQP (RabbitMQ via Docker) :

- L'API publie une tâche sur un exchange
- Le worker la consomme avec `ack`
- Gestion d'erreur, requeue et DLQ
- Back-pressure maîtrisée via `prefetch`

Durée estimée : 1h30 — Difficulté 4/5

Récapitulatif

- La back-pressure protège la mémoire : surveiller le retour de `write()` , attendre `drain` , préférer `pipeline`
- AMQP découple producteurs et consommateurs : exchanges, routing keys, ack/nack, prefetch et DLQ pour la fiabilité
- Redis Pub/Sub diffuse rapidement vers plusieurs instances ; AMQP garantit la livraison
- Choisir l'outil selon le besoin : volatile et rapide (Redis) ou fiable et acquitté (AMQP)

Cheatsheet : `cheatsheet/module-05.md`

Questions ?

Module suivant : Contrôle de qualité

Module 6 / 7